

Internet Protocol Project 1

File Transfer Performance over a Local Area Network

Project overview

This project compares file transfers between two computers on a local area network using HTTP/2 and HTTP/3. HTTP/2 uses one TCP connection to exchange files in both directions, while HTTP/3 uses one QUIC connection over UDP. We compare throughput and application overhead across four file sizes.

Team members and overall contributions

HTTP/2 contributor: Vanaja Agarwal Overall contribution: 50%

HTTP/3 contributor: Jazmin Green Overall contribution: 50%

We divided the work equally, with each member responsible for the implementation, experiments, calculations, and report section for one protocol.

Contributions by subtask

Subtask	Vanaja	Jazmin
HTTP/2 client and server implementation	100%	0%
HTTP/2 LAN experiments and integrity checks	100%	0%
HTTP/2 calculations and report section	100%	0%
HTTP/3 client and server implementation	0%	100%
HTTP/3 LAN experiments and integrity checks	0%	100%
HTTP/3 calculations and report section	0%	100%

Libraries and AI assistance

HTTP/2 libraries

We used Python and hyper-h2 version 4.3.0 [1] to build our HTTP/2 client and server. The library handles HTTP/2 frames, manages the protocol state, and compresses headers, while Python sockets send and receive data over TCP. We also used Python's standard libraries to measure time, read and write files, verify data using SHA-256, calculate statistics, and save the results.

HTTP/3 libraries

The main library used for the HTTP/3 implementation was aioquic, which provides QUIC and HTTP/3 support for Python. asyncio was used for asynchronous communication, while time was used to measure transfer times and calculate throughput. The csv and statistics libraries were used to save results and calculate throughput statistics.

AI assistance

We used Codex to help revise and test our Python HTTP/2 client and server and to check the throughput and overhead calculations. All measurements came from experiments we ran ourselves.

ChatGPT helped with the HTTP/3 setup of the client and server and troubleshooting errors during implementation. ChatGPT also created the self-signed certificate needed for the secure connection.

Experiment setup

HTTP/2 experiment setup

We connected two computers over a local area network and kept a single TCP connection open throughout the experiment. Computer 1 ran the server and stored the A files. Computer 2 first downloaded the A files using GET requests, then uploaded its own B files using POST requests over the same connection.

Each transfer used a new HTTP/2 stream. We completed one transfer before starting the next. In each direction, we transferred the 10 kB file 1,000 times, the 100 kB file 100 times, the 1 MB file 10 times, and the 10 MB file once. This added up to 1,111 downloads and 1,111 uploads. All 2,222 transfers passed both the file-size and SHA-256 checks.

We used `time.perf_counter()` [4] to measure transfer times. For downloads, timing started just before we sent the request and stopped when the full response arrived. For uploads,

timing stopped when we received the server's confirmation that the file had arrived and passed verification. The upload times therefore include the time the server spent verifying the file and sending its confirmation.

We established the connection before timing began and prepared the source files and their hashes in advance. We used cleartext HTTP/2 without TLS. On both computers, we enabled `TCP_NODELAY` and set the HTTP/2 flow-control receive window to 1 MiB. The sender stayed within the receiver's available flow-control window, following hyper-h2's guidance [2].

HTTP/3 experiment setup

For the HTTP/3 experiment, we used two computers connected over the same network, with Computer 1 acting as the client and Computer 2 acting as the server. The server listens on UDP port 4433 and waits for the client to establish a QUIC connection. The client connects to the server using the server's IP address, creating one persistent QUIC connection for all file transfers. The client uploads the A files to the server using HTTP/3 POST requests, while the B files are requested from the server using GET requests.

The experiment tests 10 kB, 100 kB, 1 MB, and 10 MB files with different numbers of transfers. During testing, the program prints the file being tested, the number of transfers, and the average and standard deviation of the throughput. The results, including file size, total transfer data, header data, application data, transfer time, and throughput, are also saved to CSV files for analysis. A self-signed certificate is used to establish the secure HTTP/3 connection between the client and server.

Throughput results

HTTP/2 throughput results

For each transfer, we divided the actual file size by the measured transfer time and converted the result to kilobits per second:

Throughput (kbps) = file size in bytes × 8 ÷ transfer time in seconds ÷ 1,000

For each file size, we combined the A and B throughput measurements and calculated the mean and sample standard deviation across all transfers. We used only the results from the full experiment over a single TCP connection and excluded the short trial runs. We rounded the values after completing the calculations.

Table 1. Mean throughput and sample standard deviation across all A and B transfers.

Size	n	Mean (kbps)	SD (kbps)
10kB	2,000	8,401.29	1,904.67
100kB	200	36,586.25	9,477.03
1MB	20	64,862.19	8,228.36
10MB	2	78,401.45	8,172.33

Average throughput increased with file size, from 8,401.29 kbps for the 10 kB files to 78,401.45 kbps for the 10 MB files. This may be because every transfer takes some time to exchange requests and responses and process the data. For larger files, these costs are spread over more bytes. The results support this explanation, but we did not measure those costs separately.

For HTTP/2, we combined the throughput measurements from the A and B transfers for each file size and calculated the sample standard deviation across all measurements. For the 10 MB files, only two transfers were available, so the reported standard deviation is based on only two observations and should not be interpreted as a reliable estimate of performance variability.

HTTP/3 throughput results

Table 2. Mean throughput and sample standard deviation across all A and B transfers.

Size	n	Mean (kbps)	SD (kbps)
10kB	2,000	5,776.83	1,499.23
100kB	200	11,586.53	4,142.11
1MB	20	13,854.08	7,298.75
10MB	2	12,992.96	9,115.90

HTTP/3 average throughput increased from 10kB to 1MB, then decreased slightly at 10MB. Because file size and transfer count changed together, the results do not show whether fewer transfers improved performance.

For HTTP/3, we combined the throughput measurements from the A and B transfers for each file size and calculated the sample standard deviation across all measurements. For the 10 MB files, only two transfers were available, so the reported standard deviation is based on only two observations and should not be interpreted as a reliable estimate of performance variability.

Application overhead

HTTP/2 application overhead

We counted the application bytes sent in the same direction as the file being transferred. This included the encoded HEADERS, CONTINUATION, and DATA frames, their nine-byte frame headers, and the file verification information. For A downloads, we counted the server's response frames. For B uploads, we counted the client's request frames. The frame format is described in [RFC 9113, Section 4.1](#).

For each transfer, we divided the application byte count by the actual file size. We then averaged these ratios across the A and B transfers for each file size:

Application data ratio = application bytes ÷ file bytes

Extra data (%) = (application data ratio - 1) × 100

A ratio of 1 means no extra data. A ratio of 1.002 means that the transfer included 0.2% extra data.

Table 3. Average application overhead across A and B transfers.

Size	Mean ratio	Extra data %
10kB	1.002203662	0.22037
100kB	1.000752490	0.07525
1MB	1.000567436	0.05674
10MB	1.000556183	0.05562

The percentage of extra data fell from 0.22037% for 10 kB files to 0.05562% for 10 MB files. Headers make up a larger share of smaller transfers, so their relative overhead decreases as file size increases. Repeated transfers can also benefit from HPACK header compression. However, some overhead remains because larger files need multiple DATA frames, each with its own nine-byte header. [RFC 9113, Sections 4.1- 4.3](#)

These measurements exclude connection setup and shared control frames, such as SETTINGS and WINDOW_UPDATE, which we recorded separately. We also excluded messages sent in the opposite direction, including download requests and upload confirmations. TCP/IP headers, link-layer headers, and retransmissions are not included in these values.

HTTP/3 application overhead

Table 4. Application overhead for A and B transfers.

Size	Average A overhead	Average B overhead
10kB	0.24%	0.28%
100kB	0.024%	0.0293%
1MB	0.0024%	0.0030%
10MB	0.00024%	0.000305%

The reported percentage of extra application data decreased as file size increased. The counted header bytes represented a smaller share of the larger files. These measurements exclude connection setup and do not include all protocol framing overhead. Because HTTP/2 and HTTP/3 counted different types of bytes, these percentages do not establish which protocol has lower overhead.

Results of HTTP/2 and HTTP/3

Throughput comparison

For each file size, we combined the A and B measurements. Both protocols had 2,000 transfers for 10 kB, 200 for 100 kB, 20 for 1 MB, and two for 10 MB. Because A and B had the same number of transfers, we calculated the combined mean by averaging their two group means.

Table 5. Average throughput across A and B transfers, in kbps.

Size	HTTP/2	HTTP/3
10kB	8,401.29	5,776.83
100kB	36,586.25	11,586.53
1MB	64,862.19	13,854.08
10MB	78,401.45	12,992.96

In our experiments, the HTTP/2 implementation recorded higher average throughput at all four file sizes. Its throughput was approximately 1.45, 3.16, 4.68, and 6.03 times the HTTP/3 throughput for 10 kB, 100 kB, 1 MB, and 10 MB files, respectively. These results describe our implementations under the tested conditions; they do not establish that HTTP/2 is generally superior. Differences in timing, encryption, and implementation may have contributed to the observed gap.

Application overhead comparison

The two implementations counted application data differently. HTTP/2 counted encoded frames sent in the same direction as the file. HTTP/3 counted decoded response headers and left out frame overhead. For uploads, HTTP/3 also counted the response headers rather than the upload headers.

Because the methods differed, we cannot directly compare the overhead values or use them to conclude which protocol has lower overhead. We would first need to count the same types of bytes in both implementations.

Limits of the comparison

These results reflect the implementations and conditions used in our tests. HTTP/2 timing started before the request was sent, while HTTP/3 timing started after its initial transmission call. HTTP/2 also ran without TLS, whereas HTTP/3 used QUIC with encryption. With these differences, we cannot attribute the entire throughput gap to the protocols themselves. [RFC 9114, Section 1.2](#)

Both experiments transferred one file at a time. This means we did not test HTTP/3's ability to keep other streams moving when one stream is waiting for missing data. That feature can help when multiple transfers run at the same time and experience packet loss, but our experiments did not cover that situation. [RFC 9114, Section 2](#)

Network conditions and the load on each computer may also have influenced the results. We tested each file size in a separate block, and the 10 MB results were based on only two transfers per protocol.

References and result sources

- [1] Python Hyper. [h2 4.3.0 package and download](#).
- [2] Python Hyper. [hyper-h2 Advanced Usage: Flow Control](#).
- [3] IETF. [RFC 9113: HTTP/2](#).
- [4] Python documentation. [time.perf_counter](#).
- [5] IETF. [RFC 9114: HTTP/3](#).
- [6] Quic. [QUIC Protocol Implementation: QUIC Protocol Implementation in Python](#).
- [7] Python Package Index (PyPI). [aioquic · PyPI](#).